MANIFIESTO MAESTRO & PLAN DE RECUPERACIÓN ANTE DESASTRES

BOT UNIVERSIDAD ONLINE

Arquitectura del Sistema, Auto-Sanación y Super Prompt Maestro de Reconstrucción Total

> **Propósito de este Documento:** Este archivo recopila formalmente la arquitectura completa del sistema de grabación desatendida, el protocolo de sincronización y auto-sanación entre la Laptop Acer y la Laptop Dell, las reglas de negocio inmutables y el **Super Prompt Maestro**. Si el equipo sufriera un formateo o pérdida de repositorios, basta con copiar y pegar dicho prompt en Google Antigravity para regenerar el 100% del ecosistema desde cero.

1. Topología del Sistema y Red Híbrida (Acer & Dell)

Dispositivo / Nodo	Dirección IP / Puerto	Rol en la Arquitectura	Procesos Ejecutados
Laptop Acer (Cliente Operativo)	192.168.100.229 Puertos 8000, 9222	Ejecución de navegador real, perfiles de sesión, captura de video y panel web.	Google Chrome (CDP 9222), Flask (8000), main.py, vigia_mentorias.py, recorder.py
Laptop Dell (Agente Antigravity IA)	192.168.100.243 Puerto 5005	Auditoría remota de código, auto-sanación, análisis heurístico y control de versiones.	Google Antigravity Agent, Endpoint de recepción /api/antigravity/sync_errores
Almacenamiento Compartido	Unidad de Red `Z:\`	Enlace SMB directo a `\\192.168.100.229\Bot_UniversidadOnline`.	clases.db, grabaciones/, logs_*.txt, registro_errores.json, config.json

2. Catálogo de Archivos y Responsabilidades de Módulos

Archivo / Módulo	Lenguaje / Tipo	Función Crítica en el Ecosistema
`main.py`	Python	Orquestador central de grabaciones, sincronización con Google Calendar, cola de post-proceso, semáforo de red y retraso de seguridad anti-RAM.
`recorder.py`	Python / Playwright	Controlador CDP de Chrome en puerto 9222. Maneja aislamiento estricto de pestañas, reproductor de Hotmart, Zoom Web ('David Herbas') y FFmpeg.

Archivo / Módulo	Lenguaje / Tipo	Función Crítica en el Ecosistema
`vigia_mentorias.py`	Python / Telethon	Monitoreo en vivo de llamadas de voz y videochats en canales de Telegram, con detector de inicio de transmisiones.
`dashboard_web.py`	Python / Flask	Panel de control web en puerto 8000. 4 consolas interactivas plegables, borrado granular (video local vs nube) y administración de mentores.
`error_buffer.py`	Python	Buffer Offline-First para registro atómico de excepciones e hilo en segundo plano que sincroniza silenciosamente cada 5 minutos hacia la Dell.
`database.py`	Python / SQLite	Capa de persistencia relacional sobre `clases.db` con bloqueo de concurrencia y funciones seguras de actualización.
`telegram_uploader.py`	Python / Telethon	Carga asíncrona de videos MP4 divididos o completos hacia los canales oficiales de Telegram con miniaturas generadas.
`notebooklm_bot.py`	Python / Playwright	Automatizador de Google NotebookLM para creación de fuentes y generación de podcasts didácticos (Audio Overviews).
`route_builder.py`	Python	Constructor del mapa interactivo de rutas de aprendizaje y sincronizador con Netlify y GitHub Pages.
`1_ENCENDIDO_MAESTRO.bat`	Batch Script	Botón maestro de encendido en un clic: mata zombies, verifica dependencias, inicia motores invisibles y lanza Chrome con CDP 9222.

3. Reglas de Negocio Inmutables (Cero Regresiones)

- **Entrada 5 Minutos Antes:** El orquestador entra 300 segundos antes a la lección para asegurar fotogramas iniciales limpios.
- **Margen Dinámico Posterior de 5 Minutos:** Grabación extendida hasta 300s post-término, reducida a 0 automáticamente si otra clase empieza de inmediato.
- **Semáforo Rojo de Red y CPU:** Prohíbe subidas y tareas pesadas durante una captura activa para erradicar cualquier congelamiento de video.
- **Aislamiento de Pestañas:** Nunca se comparten pestañas entre clases simultáneas; cada una posee un sello identificador `window.__clase_id`.
- **Salvaguarda Anti-Destrucción:** Ante fallos de remuxing, el video crudo WebM jamás se elimina y se marca como `ERROR_REMUXING`.

4. Esquema de Base de Datos (clases.db)

```
CREATE TABLE IF NOT EXISTS clases_grabadas (
    id_clase TEXT PRIMARY KEY,
    titulo_original TEXT NOT NULL,
    fecha_grabacion TEXT,
    estado TEXT NOT NULL DEFAULT 'PENDIENTE',
    ruta_archivo TEXT,
    transversal TEXT,
    nivel INTEGER DEFAULT 1,
    duracion_min INTEGER DEFAULT 60,
    importancia_creacion INTEGER DEFAULT 0,
    pareto_creacion INTEGER DEFAULT 0,
    importancia_comercializacion INTEGER DEFAULT 0,
    pareto_comercializacion INTEGER DEFAULT 0,
    analisis_json TEXT,
    notebook_url TEXT,
    podcast_archivo TEXT,
    podcast_fecha TEXT,
    url_hotmart TEXT,
    telegram_url TEXT,
    telegram_app_url TEXT,
    telegram_web_url TEXT,
    mentor TEXT DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    regrabacion_activa INTEGER DEFAULT 0
);
```

5. Super Prompt Maestro de Restauración Total (Disaster Recovery)

Copia y pega este bloque íntegro en Google Antigravity en caso de pérdida total de archivos o formateo:

```
================================================================================================
               SUPER PROMPT DE RESTAURACIÓN TOTAL - BOT UNIVERSIDAD ONLINE
================================================================================================
INSTRUCCIÓN PARA ANTIGRAVITY:
Actúa como el Arquitecto de Software Principal. He perdido mi entorno de desarrollo local y necesito
que reconstruyas el Sistema Completo del "Bot Universidad Online" exactamente con su arquitectura
empresarial, modular, resiliente y libre de regresiones.

DIRECTIVAS ARQUITECTÓNICAS CRÍTICAS:
1. PLATAFORMA Y COMUNICACIÓN:
   - Laptop Acer (Cliente Operativo: Hotmart, Chrome CDP 9222, Panel Web Flask puerto 8000).
   - Laptop Dell (Agente Antigravity / IA, IP 192.168.100.243).
   - Ruta base de trabajo: Carpeta compartida montada (Z:\) o directorio local del bot.
2. DEPENDENCIAS REQUERIDAS:
   - Python 3.8+ a 3.12, Flask, Flask-CORS, Playwright (Chromium over CDP), Telethon, requests,
     pytz, schedule, ffmpeg-python (o ejecutable ffmpeg.exe local).
3. MÓDULOS OBLIGATORIOS QUE DEBES RECREAR:
   a) 'error_buffer.py': Sistema de buffer de errores Offline-First. Intercepta excepciones no
      controladas con sys.excepthook y threading.excepthook, almacena en 'registro_errores.json'
      [id, timestamp, modulo, error_traceback, estado="Pendiente"], e incluye un hilo demonio que
      sincroniza silenciosamente cada 5 minutos vía POST a
http://192.168.100.243:5005/api/antigravity/sync_errores.
   b) 'database.py': Gestión completa de SQLite ('clases.db') con tabla 'clases_grabadas'
      y métodos para consultar pendientes, listar clases, guardar/actualizar estados y normalizar IDs.
   c) 'main.py': Orquestador concurrente con detección de horarios (-300s entrada anticipada),
      margen dinámico inteligente, semáforo de Red/CPU (ROJO durante grabación, VERDE al terminar),
      FileLogger con timestamps [HH:MM:SS] al inicio de cada línea hacia 'logs_orquestador.txt',
      y cola asíncrona de post-procesamiento.
   d) 'recorder.py': Motor Playwright conectado vía CDP a http://localhost:9222. Aislamiento estricto
      de pestañas (una por clase), búsqueda de lección en vivo en Hotmart, manejo de Zoom Web como
      'David Herbas', grabación con FFmpeg en formato crudo WebM y remuxing a MP4 con salvaguarda anti-
destrucción.
   e) 'vigia_mentorias.py': Monitoreo invisible con Telethon en segundo plano para llamadas de voz
      y videochats en canales de Telegram, con FileLogger hacia 'logs_vigia.txt'.
   f) 'telegram_uploader.py': Subida asíncrona de videos pesados a Telegram con miniaturas y captions.
   g) 'notebooklm_bot.py': Automatización de Google NotebookLM para audios de estudio y podcasts
interactivos.
   h) 'dashboard_web.py': Servidor web Flask en puerto 8000 con diseño oscuro, tabla de clases,
      borrado granular, y 4 consolas plegables (Orquestador, Vigía, Sistema y Errores e IA) con timestamps
      alineados en columna a la derecha y botones '📋 Copiar' integrados con navigator.clipboard.
   i) '1_ENCENDIDO_MAESTRO.bat': Script de encendido en un clic que mata zombies, verifica dependencias,
      arranca main.py, vigia_mentorias.py y dashboard_web.py en modo fantasma ('pythonw'), y abre
      Google Chrome con '--remote-debugging-port=9222', perfil 'perfil_chrome', cargando el panel local y
Hotmart.
   j) 'config.json': Configuración de credenciales de Telegram API, canal, webhook de calendar,
      timezones, tokens de Netlify/GitHub y parámetros de conexión hacia la Dell.

REGLAS DE NEGOCIO:
- Modo Fantasma: Procesos en segundo plano sin ventanas emergentes de consola.
- Semáforo de Red: 0% descargas/subidas pesadas mientras haya grabaciones activas.
- Zero Regressions: Toda la lógica de negocio se preserva íntegra.

Genera cada uno de estos archivos con código limpio, robusto y listo para producción inmediata.
================================================================================================
```